CLOUD & DEVOPS

Container Orchestration Compared

Kubernetes vs ECS vs Nomad — match the platform to the workload.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

The Orchestration Problem

Containers are easy; fleets of them are not.

Scheduling

which host runs this container?

Health

restart it when it dies — forever

Networking

containers find and talk to each other

Scaling

3 → 30 replicas on demand

An orchestrator is a desired-state machine for containers: you declare, it converges — like GitOps for compute.

STEP 2/6

Kubernetes: The Standard

Portable, powerful, and operationally heavy.

Kubernetes

pods, deployments, services — runs anywhere, steep learning curve

- Strengths: ecosystem (Helm, operators, service mesh), portable across clouds.
- Weaknesses: complexity — control plane, upgrades, and YAML sprawl are real work.
- Managed flavors (EKS/GKE/AKS) remove control-plane toil, not app complexity.
- Best for: multi-cloud, platform teams, complex microservices, ML pipelines.

STEP 3/6

ECS: The AWS-Native

Containers without the Kubernetes learning curve.

ECS (+ Fargate)

tasks + services — deep AWS integration, no control plane to run

- Strengths: simple mental model, IAM-native, Fargate = no servers at all.
- Weaknesses: AWS-only — no portability; smaller ecosystem than K8s.
- Fargate removes node management; EC2 launch type keeps cost control.
- Best for: AWS-committed teams, simpler service meshes, fast time-to-prod.

STEP 4/6

Nomad: The Minimalist

One binary for containers, VMs, and batch — from HashiCorp.

Nomad

jobspec → schedule — single binary, multi-workload (docker, exec, java)

- Strengths: dead simple to operate, one agent for mixed workloads (not just containers).
- Weaknesses: smaller ecosystem and community than Kubernetes.
- Pairs with Consul (discovery) + Vault (secrets): the HashiCorp stack.
- Best for: small teams, mixed workloads, edge, shops that value simplicity.

STEP 5/6

Decision Matrix

Match the platform to the workload and the team.

Multi-cloud / portable	**Kubernetes**
AWS-only, ship fast	**ECS + Fargate**
Small team, mixed workloads	**Nomad**
Complex microservices	**Kubernetes**
Batch + services together	**Nomad**
Regulated, need control	**Kubernetes (self-managed)**

STEP 6/6

Migration Advice

You can change your mind — containers are portable.

- Start with the SIMPLEST that fits: ECS/Fargate or Nomad before Kubernetes.
- Keep images registry-portable (OCI) — the orchestrator is replaceable, the image isn't.
- Kubernetes is a platform to BUILD on, not just run on — budget platform-team time.
- Re-evaluate yearly: team size, cloud commitment, and workload mix all drift.

No affiliation with any vendor implied — evaluate on your constraints: team skill, cloud lock-in tolerance, workload shape.

STEP ✓/✓

Key Takeaways

Container Orchestration Compared — Remember

- All orchestrators solve scheduling, health, networking, scaling as desired-state.
- Kubernetes: portable and powerful, operationally heavy — needs platform investment.
- ECS + Fargate: simplest AWS path, no control plane, but AWS-only.
- Nomad: minimal ops, mixed workloads, smallest ecosystem.
- Choose by constraints (team, cloud, workload) — keep images OCI-portable so you can switch.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev